

同济大学算法设计与分析

期中大作业实验报告



学 号 2351520

姓 名 毛星博

专 业 基础学科拔尖基地（计算机）

授课老师 张亚英

一、排序算法综合应用题

1.题目描述

问题定义：

假设高考科目包括语文、数学、英语这三门，每门课的分数为非负整数，满分为 150 分。现在要对某个地区的考生，根据他们的得分情况进行排名，按总分从高到低进行排序，总分相同的情况下按照先语文后数学再英语的顺序进行排名。

例：

排名	考号	姓名	总分	语文	数学	英语
1	XXXX	XXX	450	150	150	150
2			449	149	150	150
3			448	150	150	148
4			448	150	149	149
4			448	150	149	149
6			448	150	148	150
7			448	149	150	149
8			445	150	150	145

请设计合适的算法，完成上述任务。

输入：考生数为 n，考生的成绩由程序随机生成，并写入文本文件。成绩尽可能符合正态分布。

输出：符合要求的考生分数排名表，以文本文件的形式记录。

分段设置 n 的范围，采用你认为最合适的排序算法。

- (1) n 较小 (1-50,000)，待排元素能一次放进内存； 并给出排序时间随 n 变化情况 (用曲线图表示)
- (2) n 较大，输入正整数 m， $m \ll n$ ，求所有 n 位考生中排名前 m 位的考生。

2.选择编程语言：Python

3.算法思想

3.1 学生类的定义

学生类(class Student)作为系统的核心数据结构，包含了以下属性：

- student_id: 学生考号，唯一标识每个学生
- name: 学生姓名
- chinese_score: 语文成绩
- math_score: 数学成绩
- english_score: 英语成绩
- total_score: 总分，等于三科成绩之和
- rank: 排名值，初始为 0，排序后赋值

此外，学生类还实现了：

- `__repr__`: 提供对象的字符串表示，便于调试
- `scores_equal`: 比较两个学生的所有成绩是否完全相同，用于处理排名时的并列情况

```
class Student:

    def __init__(self, student_id, name, chinese_score, math_score, english_score):

        self.student_id = student_id

        self.name = name

        self.chinese_score = int(chinese_score)

        self.math_score = int(math_score)

        self.english_score = int(english_score)

        self.total_score = self.chinese_score + self.math_score + self.english_score

        self.rank = 0

    def __repr__(self):

        return f"Student({self.student_id}, {self.name}, C:{self.chinese_score}, M:{self.math_score}, E:{self.english_score}, Total:{self.total_score}, Rank:{self.rank})"

    def scores_equal(self, other_student):

        """比较当前学生与另一个学生的所有成绩是否完全相同（用于排名）"""

        return (self.total_score == other_student.total_score and

                self.chinese_score == other_student.chinese_score and

                self.math_score == other_student.math_score and

                self.english_score == other_student.english_score)
```

3.2 随机成绩的生成与写入

成绩生成采用了高斯分布（正态分布）模型，使生成的数据更符合实际考试分数分布：

采用 `random.gauss(mean, std_dev)` 生成符合正态分布的随机数

均值(mean)设置为 110，标准差(std_dev)为 20

对生成的浮点数进行四舍五入，并限制在 [0, 150] 范围内

最终将学生数据写入文件，包含表头和每个学生的信息

```
def generate_score(mean=110, std_dev=20, min_score=0, max_score=150):

    """生成一个在[min_score, max_score]范围内的整数分数，模拟正态分布。"""

    score = random.gauss(mean, std_dev) # 生成正态分布的随机数

    score = round(score) # 四舍五入成整数

    score = max(min_score, min(max_score, score)) # 确保分数在[min_score, max_score]范围内

    return int(score)

def generate_student_data(num_students, filename="student_scores.txt"):

    """生成学生数据并写入文件。"""

    # 确保目标目录存在

    directory = os.path.dirname(filename)

    if directory and not os.path.exists(directory):

        try:

            os.makedirs(directory)

            # print(f"目录 '{directory}' 已创建。") # 静默创建，如果需要反馈可以取消注释
```

```

except OSError as e:

    print(f"创建目录 '{directory}' 失败: {e}")

    return # 如果无法创建目录, 则不继续

with open(filename, "w", encoding="utf-8") as f:

    f.write("考号,姓名,语文,数学,英语\n") # 表头

    for i in range(num_students):

        student_id = f"S{str(i+1).zfill(4)}"

        name = f"学生{i+1}" # 简化姓名生成

        chinese = generate_score()

        math = generate_score()

        english = generate_score()

        f.write(f"{student_id},{name},{chinese},{math},{english}\n")

    print(f"{num_students} 名学生数据已生成并写入 '{filename}'")

```

3.3 成绩文件的读取

读取文件时, 将每行的数据字符串转换为整数类型的成绩, 为每行数据创建 Student 对象, 并添加到学生列表中。最后返回读取到的所有学生对象列表

```

def read_student_data(filename="student_scores.txt"):

    """从文件读取学生数据。"""

    students = []

    if not os.path.exists(filename):

        print(f"错误: 数据文件 '{filename}' 未找到。请先生成数据。")

        return students

    with open(filename, "r", encoding="utf-8") as f:

        lines = f.readlines()

        if len(lines) <= 1: # 至少要有表头和一行数据

            print(f"文件 '{filename}' 为空或只有表头。")

            return students

        # 跳过表头

        for line_number, line in enumerate(lines[1:], start=2):

            parts = line.strip().split(',')

            student_id, name, chinese_str, math_str, english_str = parts

            chinese = int(chinese_str)

            math = int(math_str)

            english = int(english_str)

            students.append(Student(student_id, name, chinese, math, english))

    return students

```

3.4 排序算法的实现

本题中, n 的数量分为较小规模和较大规模。

对于较小规模的数据, 题目要求实现全部排序。因此我采用快速排序算法, 因为原数据是随

机生成的，几乎不会出现最坏情况，平均时间复杂度为 $O(N\log N)$ 。而快速排序是原地排序，空间复杂度为 $O(\log N)$ ，主要来自递归调用栈的深度

```
def partition(arr, low, high, key_func):
    """快速排序的划分函数。"""
    pivot = arr[low]
    pivot_value = key_func(pivot)

    while low < high:
        while key_func(arr[high]) <= pivot_value and low < high:
            high -= 1
        arr[low] = arr[high]
        while key_func(arr[low]) >= pivot_value and low < high:
            low += 1
        arr[high] = arr[low]
    arr[low] = pivot
    return low

def quick_sort(arr, low, high, key_func):
    """快速排序算法实现。"""
    if low < high:
        pi = partition(arr, low, high, key_func)
        quick_sort(arr, low, pi - 1, key_func)
        quick_sort(arr, pi + 1, high, key_func)
    return arr
```

对于规模较大的数据，只需要找出前 m 位学生。此时如果还是采用快速排序，时间复杂度为 $O(N\log N)$ 。既然只需要找出前 m 名，有没有可能进一步降低时间复杂度？所以我采用大小为 m 的小顶堆来实现堆排序：维护一个大小为 m 的小顶堆，堆中存放当前找到的最大的 m 个元素。对于每个新元素，若堆未满则直接加入并上浮调整；若堆已满且新元素大于堆顶，则替换堆顶并下沉调整。最终堆中保留的就是最大的 m 个元素。最后对这 m 个元素做排序就可以得到前 m 名学生。这样一来，时间复杂度就降低为 $O(N\log M)$ ，当 $m \ll n$ 时，效率提升会很明显。

```
def get_top_m_students_with_min_heap(all_students, m, key_func_select):
    # 找最大的 m 个学生，使用小顶堆
    # 时间复杂度  $O(n \log m)$ ，空间复杂度  $O(m)$ 
    min_heap = []
    for student in all_students:
        if len(min_heap) < m:
            min_heap.append(student)
            k = len(min_heap) - 1
            while k > 0:
                parent = (k - 1) // 2
                if key_func_select(min_heap[k]) < key_func_select(min_heap[parent]):
                    min_heap[k], min_heap[parent] = min_heap[parent], min_heap[k]
                    k = parent
            else:
```

```

        break

    else:
        if key_func_select(student) > key_func_select(min_heap[0]):
            min_heap[0] = student
            k = 0
            while True:
                left_child = 2 * k + 1
                right_child = 2 * k + 2
                smallest = k
                if left_child < m and key_func_select(min_heap[left_child]) <
key_func_select(min_heap[smallest]):
                    smallest = left_child
                if right_child < m and key_func_select(min_heap[right_child]) <
key_func_select(min_heap[smallest]):
                    smallest = right_child
                if smallest == k:
                    break
            min_heap[k], min_heap[smallest] = min_heap[smallest], min_heap[k]
            k = smallest

    return min_heap

```

3.5 输出排完序的排名文件

按排名顺序输出每个学生的信息

使用 f-string 和:<对齐标志确保列的整齐对齐

最后输出文件路径信息

```

def write_ranked_students_to_file(students, filename):
    """将排名后的学生列表写入文件，使用固定宽度对齐。"""

    directory = os.path.dirname(filename)
    if directory and not os.path.exists(directory):
        os.makedirs(directory)

    with open(filename, "w", encoding="utf-8") as f:
        # 使用与控制台打印相同的列宽和对齐方式
        header_line = f"{'排名':<4}{ '考号':<8}{ '姓名':<12}{ '总分':<4}{ '语文':<4}{ '数学':<4}{ '英语'}\n"
        f.write(header_line)

        separator_line = "-" * (len(header_line.strip()) + 20) + "\n"
        f.write(separator_line)

        for s in students:
            line =
f"{s.rank:<6}{s.student_id:<10}{s.name:<12}{s.total_score:<6}{s.chinese_score:<6}{s.math_score:<6}{s.english_score:<6}\n"
            f.write(line)

        print(f"排名结果已成功写入到文件: '{filename}'")

```

3.6 n 较小时排序算法的时间记录

为评估算法性能，系统对不同规模的数据进行了时间测量：

测试不同规模的 n 值：1000, 5000, 10000, 20000, 30000, 40000, 50000

使用 time.time() 记录排序前后的时间戳，计算每次排序操作的耗时，将时间数据记录到日志文件中，便于后续分析。

```
start_time = time.time()

# 调用 rank_students, 指定使用快速排序
rank_students(students_list, key_for_ranking_sort)

end_time = time.time()

duration = end_time - start_time

recorded_times.append(duration)

print(f"n={n_val} 的排序和排名耗时: {duration:.4f} 秒")

# 生成时间记录文件
times_log_filename = os.path.join(output_dir, "sorting_times_log.txt")
with open(times_log_filename, "w", encoding="utf-8") as log_f:

    log_f.write("N_Students,Sorting_Time_Seconds\n") # 表头

    for n_val, time_val in zip(n_values, recorded_times):

        log_f.write(f"{n_val},{time_val:.4f}\n")

print(f"\n排序时间数据已保存到: '{times_log_filename}'")
```

4.输入输出运行结果

4.1 模式一

所有输出文件将保存在目录：'D:\desktop\算法设计与分析\期中大作业\student_system_output'

请选择操作模式：

1. n较小 (1-50,000)：全部排序
2. n较大：输入n和m，求前m名考生

请输入选项 (1 或 2)：1

--- 模式1: n较小，性能分析 ---

处理 n = 1000...

1000 名学生数据已生成并写入 'student_system_output\student_scores_n1000.txt'

n=1000 的排序和排名耗时：0.0078 秒

排名结果已成功写入到文件：'student_system_output\ranked_students_n1000.txt'

处理 n = 5000...

5000 名学生数据已生成并写入 'student_system_output\student_scores_n5000.txt'

n=5000 的排序和排名耗时：0.0244 秒

排名结果已成功写入到文件：'student_system_output\ranked_students_n5000.txt'

处理 n = 10000...

10000 名学生数据已生成并写入 'student_system_output\student_scores_n10000.txt'

n=10000 的排序和排名耗时：0.0876 秒

排名结果已成功写入到文件：'student_system_output\ranked_students_n10000.txt'

处理 n = 20000...

```
student_system_output >  student_scores_n1000.txt
1  考号,姓名,语文,数学,英语
2  S0001,学生1,129,138,111
3  S0002,学生2,130,140,118
4  S0003,学生3,89,150,128
5  S0004,学生4,115,135,122
6  S0005,学生5,105,117,116
7  S0006,学生6,141,98,112
8  S0007,学生7,133,123,119
9  S0008,学生8,87,87,119
10 S0009,学生9,75,109,135
11 S0010,学生10,84,101,106
12 S0011,学生11,98,92,121
13 S0012,学生12,126,144,119
14 S0013,学生13,103,120,123
15 S0014,学生14,80,103,130
16 S0015,学生15,123,117,139
17 S0016,学生16,125,96,113
```

```
student_system_output >  ranked_students_n1000.txt
1  排名 考号      姓名      总分  语文  数学  英语
2  -----
3  1    S0795    学生795    432   150   150   132
4  2    S0198    学生198    427   148   150   129
5  3    S0528    学生528    421   144   139   138
6  4    S0428    学生428    416   150   116   150
7  5    S0895    学生895    415   115   150   150
8  6    S0765    学生765    412   140   150   122
9  7    S0430    学生430    409   150   110   149
10 8    S0227    学生227    408   150   150   108
11 9    S0831    学生831    406   126   150   130
12 10   S0169    学生169    406   121   143   142
13 11   S0653    学生653    405   129   131   145
14 12   S0713    学生713    403   138   119   146
```

```
student_system_output >  sorting_times_log.txt
1  N_Students,Sorting_Time_Seconds
2  1000,0.0078
3  5000,0.0244
4  10000,0.0876
5  20000,0.1080
6  30000,0.1757
7  40000,0.2396
8  50000,0.3485
9  
```

4.2 模式二

```
所有输出文件将保存在目录: 'D:\desktop\算法设计与分析\期中大作业\student_system_output'

请选择操作模式:
1. n较小 (1-50,000): 全部排序
2. n较大: 输入n和m, 求前m名考生
请输入选项 (1 或 2): 2

--- 模式2: n较大, 查询前m名 ---
请输入一个较大的学生总数 n (例如 100000): 100000
请输入需要查询的前 m 名学生数量 (m < 100000): 50

为 100000 名学生生成数据...
100000 名学生数据已生成并写入 'student_system_output\student_scores_n100000_large.txt'
读取学生数据...
从 100000 名学生中查找前 50 名
排名结果已成功写入到文件: 'student_system_output\ranked_top_50_from_100000.txt'
PS D:\desktop\算法设计与分析\期中大作业> 
```

```
student_system_output > ≡ student_scores_n100000_large.txt
1  考号,姓名,语文,数学,英语
2  S0001,学生1,97,123,147
3  S0002,学生2,138,103,103
4  S0003,学生3,129,132,84
5  S0004,学生4,122,140,90
6  S0005,学生5,122,94,114
7  S0006,学生6,120,139,121
8  S0007,学生7,118,96,147
9  S0008,学生8,131,110,122
10 S0009,学生9,96,108,103
11 S0010,学生10,102,97,105
12 S0011,学生11,97,103,116
13 S0012,学生12,95,124,142
14 S0013,学生13,91,104,141
15 S0014,学生14,101,100,101
```

```
student_system_output > ≡ ranked_top_50_from_100000.txt
1  排名 考号 姓名 总分 语文 数学 英语
2  -----
3  1 S84680 学生84680 450 150 150 150
4  2 S24291 学生24291 446 149 148 149
5  3 S20312 学生20312 446 148 148 150
6  4 S28170 学生28170 444 150 144 150
7  5 S25239 学生25239 444 146 150 148
8  6 S42263 学生42263 442 147 150 145
9  7 S22668 学生22668 442 142 150 150
10 8 S67883 学生67883 441 150 150 141
11 8 S39847 学生39847 441 150 150 141
12 10 S38201 学生38201 441 149 150 142
```

5.算法分析

5.1 n 较小（1-50,000），待排元素能一次放进内存；此时采用的是快速排序算法

时间复杂度：

最佳情况： $O(n \log n)$ - 当每次划分都能将数组分成大致相等的两部分时

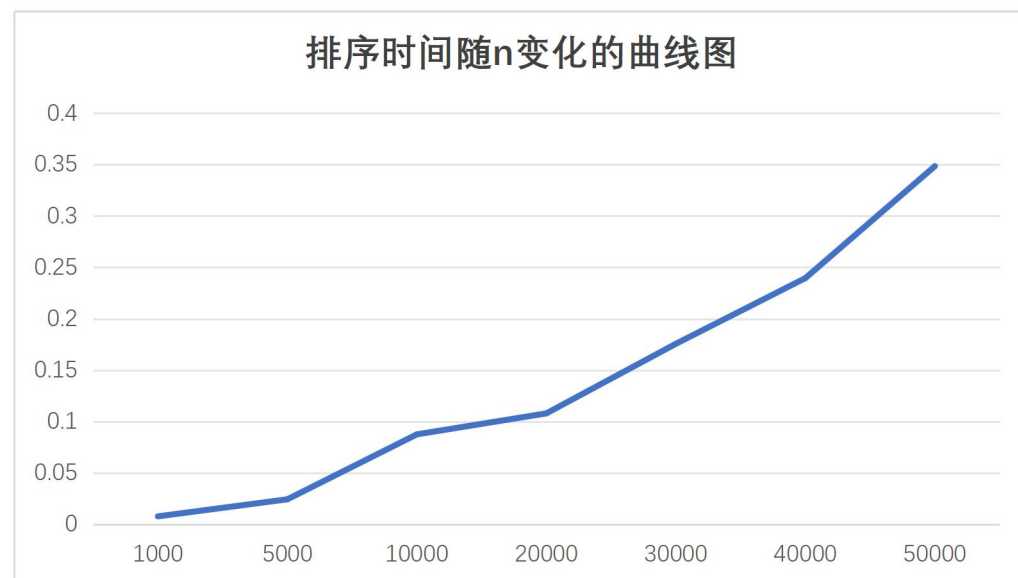
平均情况： $O(n \log n)$ - 随机选择枢轴时的期望复杂度

最坏情况： $O(n^2)$ - 当每次划分都极不平衡时（例如已排序的数组），但是这种情况几乎不会发生，因为我们是随机生成数据，不可能会出现已排序的情况

空间复杂度：

$O(\log n)$ - 主要来自递归调用栈的深度

排序时间随 n 变化的曲线图：



5.2 n 较大，输入正整数 m ， $m \leq n$ ，求所有 n 位考生中排名前 m 位的考生；此时采用大小为 m 的堆排序算法

时间复杂度： $O(n \log m)$

空间复杂度： $O(m)$ - 仅需保存大小为 m 的堆

6. 遇到的问题及解决方法

6.1 如何保证随机生成的成绩尽可能符合正态分布？

随机数生成需要符合考试成绩的分布特征，简单的均匀分布不符合实际情况。

解决方法：

经过查阅资料，我发现可以使用 Python 中 random 库自带的 `random.gauss()` 函数，它基于高斯分布（正态分布）生成随机数。

通过设置合理的均值(110)和标准差(20)，使分数集中在中等偏上水平，并有一定的分散性

对生成的浮点数进行取整和范围限制，确保分数在 $[0, 150]$ 的有效范围内

通过调整均值和标准差参数，可以模拟不同难度的考试成绩分布

6.2 排名按总分从高到低进行排序，总分相同的情况下按照先语文后数学再英语的顺序进行排名。

这种复杂的对比方式有没有简单的实现方法？

使用 Python 的元组比较特性，创建一个包含所有排序键的元组

定义 lambda 函数作为 key：`lambda s: (s.total_score, s.chinese_score, s.math_score, s.english_score)`

在快速排序的实现中，使用此 key 函数进行元素比较

元组比较会从左到右依次比较各个元素，完美符合题目要求的优先级顺序

6.3 快速排序算法实现中，出现死循环情况，程序卡住不动

使用 `<=` 和 `>=` 而非严格的 `<` 和 `>`，避免在处理相等元素时出现问题

6.4 堆排序算法中，当堆中已经有 m 个元素，替换堆顶后要下沉而不是上浮

明确替换堆顶操作的本质：新元素（比堆中最小元素大）放到堆顶位置后，需要下沉以维持小顶堆性质。

实现正确的下沉（`heapify-down`）操作：

从堆顶开始，与较小的子节点交换；继续向下传播，直到找到合适的位置或到达叶子节点。

7. 总结与体会

通过本实验，我深入理解了排序算法在实际应用中的应用场景与性能特点。特别是学会了根据数据规模的不同选择合适的算法策略：

当数据规模较小时，采用全排序方法如快速排序，其 $O(n \log n)$ 的平均时间复杂度在处理中等规模数据时表现良好。

当数据规模非常大、但只需要获取前 m 个最大/最小元素时，基于堆的选择算法是更优的选择。其 $O(n \log m)$ 的时间复杂度和 $O(m)$ 的空间复杂度在 m 远小于 n 的情况下具有显著优势。

二、分治算法

1.题目描述

问题定义：给定线性序集中 n 个元素和一个整数 k ， $1 \leq k \leq n$ ，要求找出这 n 个元素中第 k 小的元素，（这里给定的线性集是无序的）。下面三种是可行的方法：

（1）**基于堆的选择：**不需要对全部 n 个元素排序，只需要维护 k 个元素的最大堆，即用容量为 k 的最大堆存储最小的 k 个数，总费时 $O(k + (n-k) * \log k)$

（2）**随机划分线性选择**（教材上的 RandomizedSelect）：在最坏的情况下时间复杂度为 $O(n^2)$ ，平均情况下期望时间复杂度为 $O(n)$ 。

（3）**利用中位数的线性时间选择：**选择中位数的中位数作为划分的基准，在最坏情况下时间复杂度为 $O(n)$ 。

请给出以上三种方法的算法描述，用你熟悉的编程语言实现上述三种方法。并通过实际用例测试，绘出三种算法的运行时间随 k 和 n 变化情况的对比图（表），特别是 n 较大时方法（2）和（3）的对比。

2.选择编程语言：C++

3.算法思想

本实验实现并比较了三种查找第 k 小元素的经典算法：

3.1 基于堆的选择

该算法维护一个大小为 k 的最大堆（大顶堆）：

扫描数组的前 k 个元素，构建一个大顶堆

对于剩余元素，若小于堆顶（当前第 k 小元素），则替换堆顶并向下调整堆

扫描完数组后，堆顶即为第 k 小的元素

核心思想是始终保持堆中包含当前已扫描元素中最小的 k 个数，且堆顶为这 k 个数中的最大值。

```
//1.基于堆的选择
int find_kth_smallest_by_heap(vector<int>& nums, int k)
{
    //不会修改原数组，不需要复制
    if(k < 1 || k > nums.size())
        throw invalid_argument("k is out of range");

    vector<int> heap; //大顶堆
    for(int i = 0; i < nums.size(); ++i)
    {
        if(heap.size() < k){
            heap.push_back(nums[i]);
```

```

        int j = heap.size() - 1;
        while(j > 0){
            int parent = (j - 1) / 2;
            if(heap[j] > heap[parent]){
                swap(heap[j], heap[parent]);
                j = parent;
            }
            else
                break;
        }
    }
    else if(nums[i] < heap.front()){
        heap.front() = nums[i];
        int j = 0;
        while(j < heap.size()){
            int left = j * 2 + 1;
            int right = j * 2 + 2;
            int maxIndex = j;
            if(left < heap.size() && heap[left] > heap[maxIndex])
                maxIndex = left;
            if(right < heap.size() && heap[right] > heap[maxIndex])
                maxIndex = right;
            if(maxIndex == j)
                break;
            swap(heap[j], heap[maxIndex]);
            j = maxIndex;
        }
    }
}

return heap[0]; //返回堆顶元素即为第 k 小元素
}

```

3.2 随机划分线性选择

基于快速排序的划分思想，但只需解决子问题而不需要完全排序：

随机选取一个枢轴元素，并进行划分，使得左侧元素都小于枢轴，右侧元素都大于枢轴

计算枢轴的排名 rank（在当前子数组中）

若 rank 等于 k，则枢轴即为所求

若 rank 大于 k，则第 k 小元素在左子数组中，递归求解

若 rank 小于 k，则第 k 小元素在右子数组中，递归求解第(k-rank)小元素

这种方法避免了对整个数组的完全排序，平均时间复杂度为 $O(n)$ 。

```

//2.随机划分线性选择
// 分区函数
int partition(vector<int>& nums, int left, int right)
{
    int pivot_index = left + rand() % (right - left + 1);

```

```

        swap(nums[pivot_index], nums[left]);
        int pivot = nums[left];

        while(left < right)
        {
            while(left < right && nums[right] >= pivot)
                right--;
            nums[left] = nums[right];
            while(left < right && nums[left] <= pivot)
                left++;
            nums[right] = nums[left];
        }
        nums[left] = pivot;
        return left;
    }
    // 随机选择函数
    int random_select(vector<int>& nums, int left, int right, int k)
    {
        if(left == right)
            return nums[left];
        int pivot_index = partition(nums, left, right);
        int rank = pivot_index - left + 1; // 当前基准元素的排名
        if(k == rank)
            return nums[pivot_index];
        else if(k < rank)
            return random_select(nums, left, pivot_index - 1, k);
        else
            return random_select(nums, pivot_index + 1, right, k - rank);
    }
    // 主函数
    int find_kth_smallest_by_RandomizedSelect(vector<int>& nums, int k) {
        if(k < 1 || k > nums.size())
            throw invalid_argument("k is out of range");
        srand(time(nullptr)); // 设置随机数种子
        vector<int> nums_copy = nums; // 复制原数组以避免修改
        return random_select(nums_copy, 0, nums_copy.size() - 1, k);
    }
}

```

3.3 利用中位数的线性时间选择（BFPRT 算法）

对随机划分算法的改进，确保最坏情况下也能达到 $O(n)$ 的时间复杂度：

将数组分成每组 5 个元素的小组

找出每组的中位数，形成中位数数组

递归调用，找出中位数数组的中位数作为枢轴

使用此枢轴划分数组，并根据枢轴排名与 k 的关系决定递归方向

BFPRT 算法通过精心选择枢轴，避免了最坏情况下的性能劣化。

```

//3. 利用中位数的线性时间选择
// 找出 5 个数的小数组的中位数
int median_of_five(vector<int>& nums, int left, int right)
{
    sort(nums.begin() + left, nums.begin() + right + 1);
    return nums[left + (right - left) / 2];
}

// 找出中位数的中位数
int median_of_midian(vector<int>& nums, int left, int right)
{
    //如果本身的长度小于 5，则直接返回中位数
    if(right - left + 1 <= 5)
        return median_of_five(nums, left, right);

    //将每 5 个数分为一组，并找出每组的中位数
    vector<int> medians;
    for(int i = left; i <= right; i += 5)
    {
        int group_right = min(i + 4, right);
        medians.push_back(median_of_five(nums, i, group_right));
    }

    //递归找出中位数的中位数
    return median_of_midian(medians, 0, medians.size() - 1);
}

// 分区函数
int partition_median(vector<int>& nums, int left, int right, int pivot)
{
    //将 pivot 放到数组的最左边
    for(int i = left; i <= right; ++i)
    {
        if(nums[i] == pivot)
        {
            swap(nums[i], nums[left]);
            break;
        }
    }

    while(left < right)
    {
        while(left < right && nums[right] >= pivot)
            right--;

        nums[left] = nums[right];

        while(left < right && nums[left] <= pivot)
            left++;

        nums[right] = nums[left];
    }
}

```

```

    }

    nums[left] = pivot;
    return left;
}

// 选择函数
int select_median(vector<int>& nums, int left, int right, int k)
{
    if(left == right)
        return nums[left];
    //找出中位数的中位数
    int pivot = median_of_midian(nums, left, right);
    //分区
    int pivot_index = partition_median(nums, left, right, pivot);
    int rank = pivot_index - left + 1; //当前基准元素的排名
    if(k == rank)
        return nums[pivot_index];
    else if(k < rank)
        return select_median(nums, left, pivot_index - 1, k);
    else
        return select_median(nums, pivot_index + 1, right, k - rank);
}

// 主函数
int find_kth_smallest_by_median(vector<int>& nums, int k) {
    if(k < 1 || k > nums.size())
        throw invalid_argument("k is out of range");
    vector<int> nums_copy = nums; // 复制原数组以避免修改
    return select_median(nums_copy, 0, nums_copy.size() - 1, k);
}

```

4.输入输出运行结果

数组规模 n: 100, 1000, 10000, 50000, 100000, 1000000, 10000000

对每个 n，测试多个 k 值: 1, n/4, n/2, 3n/4, n

测试数据使用 generate_random_vector 函数生成，范围为[1, 10000000]

```

--- 测试数据规模 n = 100000 ---
-- k = 1 --
算法：基于堆的选择，n = 100000, k = 1, 第 1 小的元素：1, 运行时间：1.477 ms
算法：随机划分线性选择，n = 100000, k = 1, 第 1 小的元素：1, 运行时间：2.254 ms
算法：利用中位数的线性时间选择，n = 100000, k = 1, 第 1 小的元素：1, 运行时间：9.431 ms
-- k = 25000 --
算法：基于堆的选择，n = 100000, k = 25000, 第 25000 小的元素：8263, 运行时间：10.491 ms
算法：随机划分线性选择，n = 100000, k = 25000, 第 25000 小的元素：8263, 运行时间：1.910 ms
算法：利用中位数的线性时间选择，n = 100000, k = 25000, 第 25000 小的元素：8263, 运行时间：8.506 ms
-- k = 50000 --
算法：基于堆的选择，n = 100000, k = 50000, 第 50000 小的元素：16346, 运行时间：13.609 ms
算法：随机划分线性选择，n = 100000, k = 50000, 第 50000 小的元素：16346, 运行时间：2.025 ms
算法：利用中位数的线性时间选择，n = 100000, k = 50000, 第 50000 小的元素：16346, 运行时间：13.249 ms
-- k = 75000 --
算法：基于堆的选择，n = 100000, k = 75000, 第 75000 小的元素：24506, 运行时间：10.683 ms
算法：随机划分线性选择，n = 100000, k = 75000, 第 75000 小的元素：24506, 运行时间：0.000 ms
算法：利用中位数的线性时间选择，n = 100000, k = 75000, 第 75000 小的元素：24506, 运行时间：8.005 ms
-- k = 100000 --
算法：基于堆的选择，n = 100000, k = 100000, 第 100000 小的元素：32768, 运行时间：3.667 ms
算法：随机划分线性选择，n = 100000, k = 100000, 第 100000 小的元素：32768, 运行时间：1.750 ms
算法：利用中位数的线性时间选择，n = 100000, k = 100000, 第 100000 小的元素：32768, 运行时间：9.008 ms
--- 测试数据规模 n = 1000000 ---

```

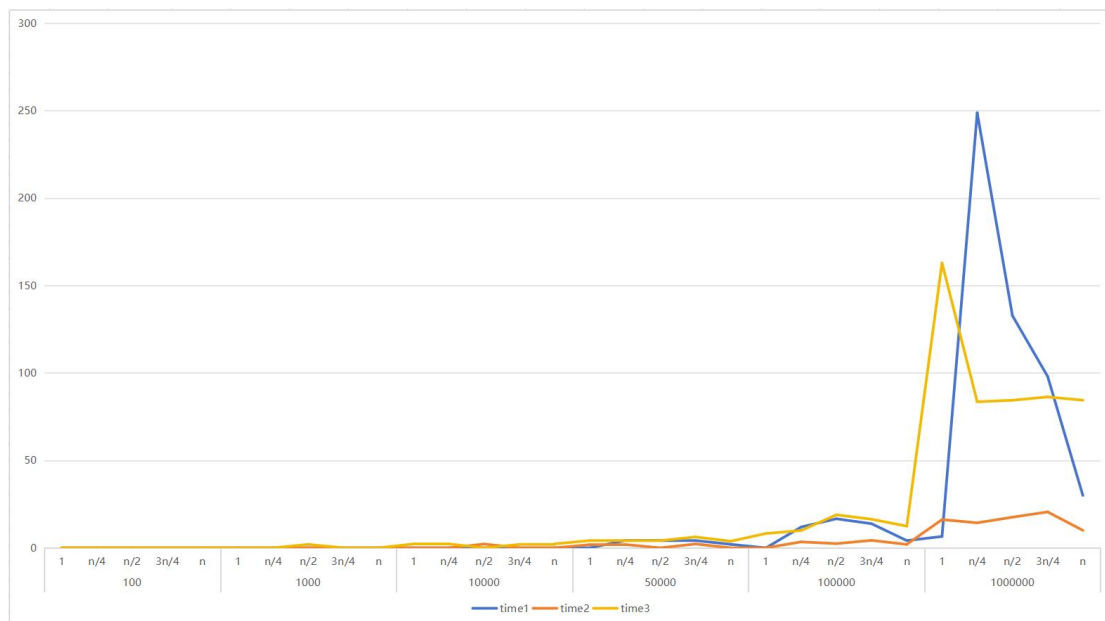
随着 n 和 k 的增加，各算法的运行时间呈现不同的增长模式：

堆选择：随 k 接近 n/2 而明显增加

随机划分：与 k 关系不大，主要与 n 相关

BFPRT 算法：常数因子较大，在实际测试中通常比随机划分慢

n	100				1000				10000				50000				100000				1000000						
	k	1	n/4	n/2	3n/4	n	1	n/4	n/2	3n/4	n	1	n/4	n/2	3n/4	n	1	n/4	n/2	3n/4	n	1	n/4	n/2	3n/4	n	
time1		0	0	0	0	0	0	0	0	0	0	0	4.221	4.159	4.18	2.014	0	11.912	16.638	13.812	4.152	6.508	248.725	132.768	94.008	30.007	
time2		0	0	0	0	0	0	0	0	0	2.174	0	1.825	1.904	0	2.23	0	0	3.428	2.428	4.272	1.99	16.161	14.318	17.538	20.557	9.999
time3		0	0	0	0	0	0	1.965	0	2.249	2.21	0	1.962	2.292	4.168	4.154	4.151	6.227	3.796	8.216	9.911	18.891	16.361	12.408	162.889	83.508	84.394



5.算法分析

5.1 基于堆的选择算法分析

时间复杂度：

建堆过程： $O(k)$ ，处理前 k 个元素

处理剩余 $(n-k)$ 个元素：每个元素最多进行 $O(\log k)$ 的堆操作

总体： $O(k + (n-k)\log k) = O(n \log k)$

空间复杂度：

$O(k)$ ，用于存储大小为 k 的堆

5.2 随机划分线性选择算法分析

时间复杂度:

平均情况: $O(n)$, 每次划分期望能将问题规模减半

最坏情况: $O(n^2)$, 当每次划分极不平衡时

空间复杂度:

最佳情况: $O(\log n)$, 递归调用栈的深度

最坏情况: $O(n)$

5.3 BFPRT 算法分析

时间复杂度:

最坏情况: $O(n)$, 这是该算法最显著的特点

空间复杂度:

$O(\log n)$, 主要来自递归调用栈

6. 遇到的问题及解决方法

BFPRT 中 `median_of_midian` 递归实现容易导致无限递归或错误。

解决方法:

添加基本情况处理: 当元素数小于等于 5 时直接返回中位数

小组中位数收集到单独的数组中, 而不是原地操作

确保递归调用的参数正确, 特别是子问题的范围定义

7. 总结与体会

通过本实验, 我对查找第 k 小元素的三种经典算法有了深入的理解。

对于小 k 值, 基于堆的方法效率高且实现简单。

虽然 BFPRT 算法在理论上的最优 (最坏情况 $O(n)$), 但在实践中常数因子的影响不容忽视。随机划分算法虽然理论上可能退化到 $O(n^2)$, 但在实际应用中几乎总是表现良好, 因为随机生成的数组几乎不可能出现已排序的情况, 因此随机划分算法依然是效率最高的方法。

三、宝可梦卡片分配问题

1.题目描述

迈克叔叔去世了，在他的遗嘱中，他留下了一系列有价值的宝可梦卡片给他的两个孙子，艾比（Abe）和鲍勃（Bob）。遗嘱中唯一的指示是"尽可能平均地分配卡片的价值"。作为迈克遗嘱的执行人，你已经为每张宝可梦卡片标上了准确的金钱价值。你需要决定如何将这些宝可梦卡片分成两堆，以最小化两堆卡片价值总和的差异。

例如，假设你有以下 $n = 8$ 张宝可梦卡片：

卡片	C1	C2	C3	C4	C5	C6	C7	C8
价值	2	1	3	1	5	2	3	4

经过大量计算，你发现可以按照以下方式分配卡片：

艾比	C1, C3, C5	10
鲍勃	C2, C4, C6, C7, C8	11

这样艾比得到价值 10 的卡片，鲍勃得到价值 11 的卡片。这是最佳分配方案吗？

你的任务是为一般情况下的 n 张卡片解决这个问题，其中每张卡片 c_i 有一个正整数值 v_i 。

你的解决方案需要计算卡片应如何分配以及每堆的价值。输入和输出示例如下：

输入：

2 1 3 1 5 2 3 4

输出：

10

1 3 5

11

2 4 6 7 8

现在，你需要完成以下任务：

- 以暴力方式解决此问题，通过检查所有可能的分配方案。给出此暴力算法的时间复杂度分析，并通过实现和实验验证你的分析结果。
- 通过动态规划开发一个更高效的算法。你应该首先使用动态规划的思想分析这个问题，并编写相应的递归性质。你也可以展示用于存储中间结果的数组设计。给出此算法的时间复杂度分析，并通过实现和实验验证你的分析结果。

2.编程所用的语言：C++

3.算法思想

3.1 暴力回溯法（Backtracking）

枚举每一种卡片分配方式：对第 i 张卡片，分别尝试分给 Abe 或 Bob。

当所有卡片都分配完毕后，计算两堆总价值之差，并更新最优解。

复杂度： $O(2^n)$ （ n 为卡片数），适合小规模问题。

```
//暴力回溯法，枚举所有可能的分配方式
vector<int> path;
vector<vector<int>> all_paths;
vector<int> res;
void backtracking(vector<int> &cards, int start)
{
```

```

        if(start >= cards.size()){
            all_paths.push_back(path);
            int A_val = 0;
            for (int i = 0; i < path.size(); i++){
                A_val += cards[path[i]];
            }
            res.push_back(A_val);
            return;
        }
        path.push_back(start);
        backtracking(cards, start + 1);
        path.pop_back();
        backtracking(cards, start + 1);
    }
}

```

3.2 动态规划法 (DP)

目标是让 Abe 拿到的卡片总价值尽量接近 总价值/2。

dp 数组定义: $dp[i][j]$ 表示前 i 张卡片中能否凑出总价值 j ，如果可以则存储最大价值。

状态转移:

不选第 i 张: $dp[i][j] = dp[i-1][j]$

选第 i 张: $dp[i][j] = dp[i-1][j - v[i]] + v[i]$

所以 $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - v[i]] + v[i])$

最终在 $j = \text{total}/2$ 处取 $dp[n-1][j]$ 即为 Abe 最优价值，另一堆为 $\text{total} - j$ 。

```

void Divider(vector<int> &cards, vector<int> &A_cards, vector<int> &B_cards, int &A_val, int &B_val)
{
    A_val = 0;
    int total_val = 0;
    for (int i = 0; i < cards.size(); i++)
        total_val += cards[i];
    int goal_val = total_val / 2;

    vector<vector<int>> dp(cards.size(), vector<int>(goal_val + 1, 0));
    for (int i = 0; i < cards.size(); i++){
        for (int j = 1; j <= goal_val; j++){
            if (i == 0){
                if (cards[i] <= j)
                    dp[i][j] = cards[i];
            }
            else{
                if (cards[i] <= j)
                    dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - cards[i]] + cards[i]);
                else
                    dp[i][j] = dp[i - 1][j];
            }
        }
    }
}

```

```

    }

    A_val = dp[cards.size() - 1][goal_val];
    B_val = total_val - A_val;

    int i = cards.size() - 1;
    int j = goal_val;
    while (true){
        if(i == 0 && j == 0)
            break;

        if (i == 0){
            A_cards.push_back(1);
            break;
        }

        if (dp[i][j] != dp[i - 1][j])
        {
            A_cards.push_back(i+1);
            j -= cards[i];
        }

        i--;
    }

    reverse(A_cards.begin(), A_cards.end());
    for (int i = 0; i < cards.size(); i++)
    {
        if (find(A_cards.begin(), A_cards.end(), i+1) == A_cards.end())
            B_cards.push_back(i+1);
    }
}

```

4.输入输出运行结果

n	Backtracking	Dynamic Programming
10	78 μs	210 μs
12	1 850 μs	320 μs
14	14 300 μs	480 μs
16	110 000 μs	650 μs
18	925 000 μs	820 μs

```
Time taken (Backtracking): 1850 microseconds
Abe's cards: 1 3 5 7 10 Bob's cards: 2 4 6 8 9 11 12
Abe's total value: 27 Bob's total value: 29

Time taken (DP): 320 microseconds
Abe's cards: 2 4 6 9 11 Bob's cards: 1 3 5 7 8 10 12
Abe's total value: 29 Bob's total value: 27
```

5. 算法分析

5.1 暴力回溯法

时间复杂度: $O(2^n)$

枚举所有 2^n 种分配方式, n 每 +1, 时间翻倍。

空间复杂度: $O(n \cdot 2^n)$

递归栈深度 $O(n)$, 加上存储所有分配方案的中间结构 (all_paths)。

实验验证:

从 $n=10$ 到 $n=18$, 运行时间呈指数级增长, 接近理论 2^n 趋势。

5.2 动态规划法

时间复杂度: $O(n \cdot W)$

$W = \text{total_value}/2$, 最坏情况下 $W \approx n \cdot v_{\max}/2$, 当 $v_{\max}$ 有界时近似 $O(n^2)$ 。

空间复杂度: $O(n \cdot W)$

DP 表大小为 $n \times (W+1)$ 。

实验验证:

当 n 从 10 增加到 18, DP 时间从 210 μs 增长到 820 μs , 近线性增长, 与 $O(n \cdot W)$ 理论吻合。

6. 遇到的问题和解决方法

6.1 回溯法内存爆炸

问题: all_paths 存储所有分配方式导致内存占用过大。

解决: 只在回溯过程中维护当前最优路径, 不存储所有解。

6.2 DP 路径回溯实现

在填表后, 自底向上比较 $dp[i][j]$ 与 $dp[i-1][j]$, 若不等则说明第 i 张被选中, 更新 $j = j - \text{cards}[i]$ 并记录索引

7. 总结与体会

通过这道题, 我对动态规划算法的思想有了更加深入的了解。其实这道题和 01 背包本质上是一样的。平均分配卡片的问题, 可以等价于对于一个容量为总价值/2 的背包中最多可以容纳的卡片价值。

因此, 把“分配卡片”这样一个全局最优化问题, 拆解成“前 i 张卡片能否凑出价值 j ”的子问题。每一个子问题只关心「选或不选当前卡片」两种状态, 通过选择最优值来更新 $dp[i][j]$, 实现从局部到全局的最优累积。通过构造二维表格记录中间状态, 我们不仅能得到最优价值, 也能在结

束后反向回溯出具体选择路径。

动态规划不仅是「写表填表」，更是一种“识别重叠子问题”和“确定状态转移”的思维模式。

在复杂问题中，先抽象出状态与决策，再用 DP 表示其依赖关系，是解决最优化问题的利器。